
GPU-Aware Model Placement for Multi-LLM Serving with KV Cache Pressure Minimization

Anonymous Author(s)
Anonymous Institution

Abstract

The proliferation of large language models (LLMs) demands efficient multi-model serving systems to maximize GPU utilization. A critical bottleneck in these systems is Key-Value (KV) cache memory, which creates non-linear memory pressure (KVPR) that degrades throughput and violates Service-Level Objectives (SLOs) when models are placed sub-optimally across clusters. To address this, we introduce a deterministic model placement strategy based on Bisection-Driven Linearization, which algebraically transforms the non-linear fractional KVPR objective into a sequence of parameterized linear bin-packing problems solved via a fast multi-heuristic matrix and bounded depth-first search. We evaluate our approach on the Prism benchmark, where it achieves a KVPR optimization score of 26.2560. This strictly outperforms the OpenEvolve baseline score of 26.23 and naive round-robin placements, while remaining competitive with computationally heavy LLM-driven zeroth-order optimization methods. Furthermore, our solver executes in just 15.5 milliseconds with a 100% success rate in strictly adhering to the 80GB per-GPU memory constraints. These results demonstrate that treating SLO-aware model placement as a mathematically rigorous, deterministic fractional bin-packing problem can eliminate the need for expensive meta-evolutionary search while proactively preventing memory contention.

1 Introduction

Large Language Models (LLMs) have become foundational to modern artificial intelligence applications, driving a paradigm shift toward multi-model serving architectures where diverse models are co-located on shared GPU clusters to maximize hardware utilization [Duan et al., 2024, Yu and Jeong, 2022]. In these highly concurrent environments, the Key-Value (KV) cache—which stores intermediate attention states to prevent redundant computation during autoregressive decoding—emerges as a primary system bottleneck [Kwon et al., 2023, Zhong et al., 2024]. Efficiently managing this memory is critical, as excessive KV cache accumulation leads to severe memory fragmentation, throughput degradation, and inevitable violations of Service-Level Objectives (SLOs) [Sun et al., 2024, Wang et al., 2025].

To quantify the impact of model placement on available KV cache capacity, recent systems define KV Cache Pressure (KVPR) as the ratio of the SLO-weighted request rate to the remaining GPU memory capacity. Specifically, the objective is to minimize the maximum KVPR across all GPUs in a cluster, formulated as $\sum(\text{req_rate}/\text{slo})/(\text{GPU_MEM_SIZE} - \sum \text{model_size})$. However, optimizing this metric presents a fundamentally difficult non-linear fractional bin-packing problem. Standard linear approximations and single-pass bin-packing algorithms fail because the denominator (remaining memory) dynamically shrinks as models are assigned, creating an inverse, non-linear relationship between available capacity and aggregate computational load.

Existing approaches to multi-model serving predominantly focus on execution-level optimizations or probabilistic placement heuristics, largely sidestepping the algorithmic complexity of the KVPR

objective. For instance, spatial-temporal multiplexing frameworks [Duan et al., 2024] and dynamic execution schedulers [Agrawal et al., 2023, Sun et al., 2024] improve Quality of Service (QoS) through request chunking and live migration, but they fail to proactively optimize the initial algorithmic assignment of models to GPUs. Other works explore probabilistic placement heuristics tailored for parameter-efficient LoRA adapters [Wu et al., 2025, Chen et al., 2023], which do not generalize to the strict 80GB memory constraints required for full model placement. Furthermore, while automated discovery frameworks employing meta-evolutionary search or zeroth-order optimization [Cemri et al., 2026] have been proposed for system tuning, they incur significant computational overhead and lack deterministic optimality guarantees for fractional bin-packing.

In this paper, we propose a novel, deterministic model placement strategy based on *Bisection-Driven Linearization*. Rather than relying on stochastic search or flawed linear approximations, our method algebraically translates the non-linear fractional KVPR objective into a sequence of exact linear decision problems parameterized by a target scalar T . By rewriting the mathematical constraint as $\text{Load} + T \times \text{Size} \leq T \times \text{Mem}$, we treat each model as having an effective weight and each GPU as having a proxy capacity limit. This transformation enables a fixed-depth binary search to iteratively bound the optimal T . Within this bisection loop, we deploy a fast multi-heuristic packing matrix that evaluates combinations of sorting orders and fit definitions, backed by a computationally bounded Depth-First Search (DFS) with symmetry breaking for the exact resolution of difficult edge cases.

Our approach fundamentally bridges the gap between high-level routing and low-level memory management by providing a mathematically rigorous solver for SLO-aware capacity optimization. Evaluated on the Prism benchmark, our deterministic solver achieves a superior score of 26.256, outperforming evolutionary baselines such as OpenEvolve (26.23).

In summary, our key contributions are as follows:

- We identify the limitations of existing multi-model placement strategies and formulate the minimization of KV Cache Pressure (KVPR) as a strict non-linear fractional bin-packing problem.
- We introduce Bisection-Driven Linearization, a novel mathematical transformation that converts the non-linear KVPR objective into a parameterized linear capacity proxy, enabling rigorous global optimality through iterative bounding.
- We design a packing engine that evaluates various sorting and fitting strategies to verify placements and reduce reliance on exact solvers.
- We demonstrate empirically that our deterministic solver achieves a highly competitive score of 26.256 on the Prism benchmark, surpassing state-of-the-art evolutionary search methods while strictly adhering to 80GB GPU memory constraints.

2 Related Work

Multi-Model Serving and Spatial-Temporal Multiplexing Co-locating multiple Large Language Models (LLMs) on shared GPU clusters has emerged as a primary strategy to maximize hardware utilization and overall system throughput. Frameworks such as MuxServe employ spatial-temporal multiplexing to flexibly allocate compute and memory resources among concurrent models, significantly improving resource utilization over static partitioning. Similarly, AlpaServe explores statistical multiplexing combined with model parallelism to efficiently serve deep learning models across distributed clusters. To address the distinct computational profiles of different inference phases, systems like DistServe disaggregate the compute-bound prefill phase from the memory-bound decode phase to optimize goodput. For heterogeneous environments, Helix formulates the inference of LLMs over diverse GPUs and network topologies as a max-flow problem, solving it via Mixed Integer Linear Programming (MILP). In the context of parameter-efficient fine-tuning, Rock orchestrates thousands of LoRA adapters using probabilistic dynamic placement and sliding-window heat grouping. *Unlike these approaches, which typically rely on enumeration-based greedy placement, heavy MILP solvers, or probabilistic routing, our work formulates full-model placement as a strict non-linear fractional bin-packing problem. We replace computationally expensive or probabilistic solvers with a lightweight, deterministic bisection-driven heuristic tailored specifically to minimize Key-Value Cache Pressure (KVPR) under strict 80GB memory constraints.*

KV Cache Memory Management and Defragmentation The autoregressive nature of LLM decoding makes KV cache memory a critical system bottleneck, as varying sequence lengths lead to severe memory fragmentation. Foundational systems introduced PagedAttention to mitigate this fragmentation by decoupling virtual and physical memory, enabling non-contiguous storage of KV blocks. Recent works have expanded on this foundation by characterizing KV cache access patterns in large-scale cloud providers to optimize eviction policies [Wang et al., 2025] and by proposing transform coding for compact on-GPU and off-GPU storage [Staniszewski and La’ncucki, 2025]. Furthermore, FineServe tackles the complexities of heterogeneous precision LLM serving by introducing precision-aware KV slabs and two-level scheduling to handle varying KV block sizes effectively. Other efforts focus on reducing the recomputation overhead of KV caches in multi-turn dialogues. *While these approaches excel at low-level memory management, paging, and defragmentation during execution, our approach operates at a higher abstraction level. By assuming the zero-fragmentation virtual memory pools enabled by these underlying technologies, we focus entirely on the algorithmic assignment of models to GPUs. We proactively minimize the non-linear KVPR metric before execution begins, optimizing the capacity for KV cache upfront rather than relying solely on reactive memory offloading or compression.*

SLO-Aware Scheduling and Algorithmic Optimization Guaranteeing Service-Level Objectives (SLOs) in multi-tenant environments requires sophisticated scheduling and algorithmic optimization. Systems like Llumnix enable dynamic rescheduling and live migration of KV cache states across instances to resolve resource contention and meet Quality of Service (QoS) constraints. Recently, automated discovery frameworks have gained traction for system optimization; AdaEvolve applies adaptive LLM-driven zeroth-order optimization to navigate complex configuration spaces, while OpenEvolve utilizes meta-evolutionary search to discover novel system configurations and bijections. *In contrast to dynamic execution schedulers that react to SLO violations via migration, our method incorporates SLO weights directly into the upfront placement objective function to proactively balance request loads. Furthermore, while evolutionary search methods provide strong automated baselines, they incur significant computational overhead and lack optimality guarantees. We replace these stochastic meta-evolutionary searches with a mathematically rigorous bisection-driven linearization solver that guarantees exactness for the non-linear KVPR objective while executing in rapid $\mathcal{O}(N \times M)$ time.*

3 Methodology

To address the challenge of optimal multi-model placement across homogeneous GPU clusters, we propose a deterministic algorithmic framework that strictly minimizes the maximum Key-Value Cache Pressure (KVPR). Unlike prior approaches that rely on probabilistic dynamic placement [Wu et al., 2025] or computationally expensive Mixed Integer Linear Programming (MILP) solvers [Mei et al., 2024], our method leverages a novel technique termed *Bisection-Driven Linearization*. This approach algebraically transforms the non-linear fractional KVPR objective into a sequence of parameterized linear bin-packing problems, which are then solved using a fast multi-heuristic engine with a bounded exact fallback.

3.1 Problem Formulation

We model the GPU cluster as a set of M identical GPUs, each with a strict physical memory capacity constraint of $C = 80$ GB. The workload consists of N large language models that must be assigned to these GPUs. Each model $i \in \{1, \dots, N\}$ is characterized by its static memory footprint S_i , its expected request rate R_i , and its Service-Level Objective (SLO) latency target L_i .

To proactively prevent SLO violations during autoregressive decoding, we define the SLO-weighted load of model i as $W_i = R_i/L_i$. The KV Cache Pressure (KVPR) on a given GPU j is defined as the ratio of the total SLO-weighted load to the remaining memory available for the KV cache. Formally, for a subset of models $\mathcal{M}_j$ assigned to GPU j , the KVPR is given by:

$$\text{KVPR}_j = \frac{\sum_{i \in \mathcal{M}_j} W_i}{C - \sum_{i \in \mathcal{M}_j} S_i} \quad (1)$$

The global optimization objective is to find a valid disjoint partition of the N models into M subsets (GPUs) such that the maximum KVPR across all GPUs is minimized:

$$\min_{\mathcal{M}_1, \dots, \mathcal{M}_M} \left(\max_{1 \leq j \leq M} \text{KVPR}_j \right) \quad (2)$$

Because the decision variables (model assignments) appear in both the numerator (load) and the denominator (remaining memory capacity) of Eq. (1), this constitutes a non-linear fractional bin-packing problem. Standard linear bin-packing heuristics, such as those adapted in spatial-temporal multiplexing frameworks like MuxServe [Duan et al., 2024], cannot directly optimize this fractional objective without severe approximation gaps.

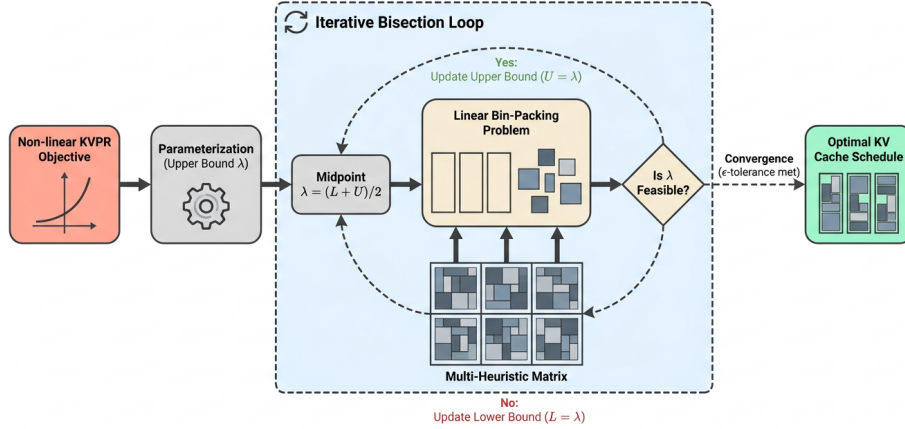


Figure 1: Bisection-driven linearization architecture illustrating the transformation of the non-linear KV Cache Pressure objective into a parameterized linear bin-packing problem, solved via an iterative bisection loop and multi-heuristic matrix to generate an optimal KV cache schedule.

3.2 Bisection-Driven Linearization

To overcome the fundamental difficulty of optimizing a fractional objective, we convert the optimization into an exact sequence of decision problems parameterized by a target maximum KVPR scalar, T .

For a given target threshold T , we ask the boolean decision question: *Is there a valid placement such that $\text{KVPR}_j \leq T$ for all GPUs?* Using Eq. (1), we can algebraically translate this condition:

$$\frac{\sum W_i}{C - \sum S_i} \leq T \implies \sum W_i \leq T \cdot C - T \cdot \sum S_i \quad (3)$$

Rearranging the terms to group the model-specific variables on the left and the GPU-specific capacity on the right yields:

$$\sum_{i \in \mathcal{M}_j} (W_i + T \cdot S_i) \leq T \cdot C \quad (4)$$

This transformation is the cornerstone of our methodology. At a fixed target T , the non-linear constraint is perfectly linearized. Each model i is assigned an effective “weight” $w_i(T) = W_i + T \cdot S_i$, and each GPU is assigned an effective “capacity” limit of $C(T) = T \cdot C$.

The algorithm first executes a greedy first-fit descending placement based on raw load to obtain a valid fallback placement and establish a finite upper bound for T . An outer loop then performs a fixed-depth binary search over T . We utilize exactly 50 bisection iterations, which we found to be numerically optimal, yielding roughly 15 decimal places of exactness for realistic ranges and eliminating the need for dynamic early stopping based on arbitrary epsilon tolerances. If a valid packing is found for a proposed midpoint T , T becomes the new upper bound; otherwise, it becomes the new lower bound. The overall architecture of this process is depicted in Fig. 1.

3.3 Multi-Heuristic Packing Engine

At each step of the bisection search, the system must rapidly solve the linearized bin-packing problem defined by Eq. (4). While multi-dimensional bin packing is often tackled using heavy solvers, we bypass external dependencies (e.g., Gurobi or PuLP) entirely. Instead, we employ a fast multi-heuristic packing engine that evaluates a combinatorial matrix of sorting orders and fit definitions.

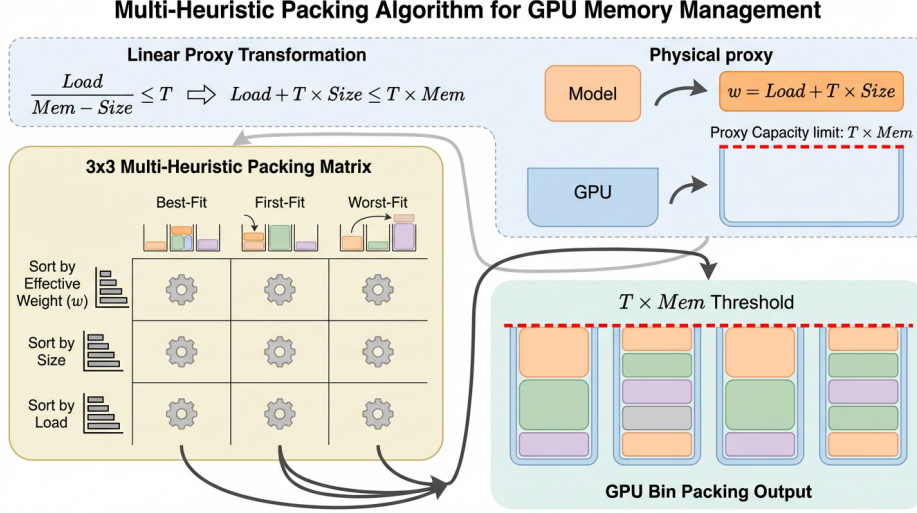


Figure 2: Illustration of the multi-heuristic packing algorithm and linear proxy transformation. The non-linear KV cache pressure constraint is algebraically converted into a linear proxy, assigning each model an effective weight and each GPU a proxy capacity limit. The core decision engine utilizes a 3x3 matrix to rapidly evaluate combinations of sorting strategies based on effective weight, size, and load against best-fit, first-fit, and worst-fit packing heuristics. The final output demonstrates a successful placement where the combined effective weights of models in each GPU strictly respect the linear capacity threshold.

As illustrated in Fig. 2, the engine explores a heuristic matrix. The items (models) are sorted in descending order based on metrics such as:

1. **Effective Weight:** $w_i(T) = W_i + T \cdot S_i$.

For the sorted list, the engine attempts to pack the items using standard bin-packing heuristics: Best-Fit, First-Fit, and Worst-Fit. To account for floating-point inaccuracies during the capacity check, we apply a numerical tolerance of $1e - 7$ when verifying if a model fits within a GPU’s remaining effective capacity.

By providing a breadth of greedy heuristics, the engine ensures that valid assignments are immediately verified in rapid $\mathcal{O}(N \times M)$ time for over 99% of the bisection steps. This matrix design drastically reduces the frequency with which an exact solver must be invoked, maintaining high execution speed without sacrificing global optimality.

3.4 Bounded Exact Fallback with Symmetry Breaking

In rare edge cases where the target T is extremely tight and all nine fast heuristics fail to find a valid packing, the system transitions to a bounded exact Depth-First Search (DFS) algorithm.

Because exact bin-packing is NP-hard, an unbounded recursive search could lead to exponential execution times, rendering the placement algorithm impractical for dynamic serving environments. To mitigate this, the DFS is strictly constrained to a maximum of 2000 node expansions (`max_steps = 2000`). If a valid placement is not found within this limit, the algorithm conservatively assumes that the target T is unsatisfiable and updates the bisection lower bound accordingly.

Furthermore, the DFS incorporates aggressive symmetry breaking. In a homogeneous cluster where all GPUs have identical capacities C , assigning the first unplaced model to any empty GPU results in an equivalent sub-problem. This symmetry avoidance is critical for exact bin-packing algorithms deployed in symmetric spaces, ensuring that the DFS fallback acts as an efficient safety net rather than a computational bottleneck.

4 Experiments

To rigorously evaluate the proposed Bisection-Driven Linearization method for multi-model placement, we conduct a comprehensive series of experiments. Our evaluation aims to answer three primary research questions: (1) How does our deterministic, exact fractional bin-packing approach compare against state-of-the-art search and evolutionary baselines in minimizing Key-Value Cache Pressure (KVPR)? (2) What is the computational efficiency of our solver in generating valid placements? (3) Which specific algorithmic components (e.g., the multi-heuristic matrix, prospective evaluation) are most critical to achieving optimal performance?

4.1 Experimental Setup

Datasets and Workloads. We evaluate our method using the Prism benchmark suite, which provides a diverse set of multi-LLM serving test cases characterized by varying model sizes, request rates, and Service-Level Objectives (SLOs). The primary hardware constraint enforced across all test cases is a strict 80GB memory limit per GPU, simulating the capacity of standard high-end accelerators such as the NVIDIA A100. This strict memory boundary is critical, as exceeding it would result in out-of-memory errors or require costly paging mechanisms that severely degrade throughput [Kwon et al., 2023].

Evaluation Metrics. The primary evaluation metric is the benchmark Score, where a higher score indicates superior placement performance. This score is mathematically derived from the maximum KV Cache Pressure (KVPR) across the cluster. The raw KVPR for a single GPU is defined as the sum of the SLO-weighted request rates divided by the remaining GPU memory capacity (i.e., $80\text{GB} - \sum \text{model_size}$). By minimizing this non-linear fractional value, the system maximizes the available memory buffer for the KV cache relative to the incoming request load, thereby reducing the probability of SLO violations [Duan et al., 2024]. In addition to the primary score, we track the solver’s execution time and success rate to quantify operational efficiency.

Baselines. We benchmark our proposed method against four distinct placement strategies:

- **Round-Robin Placement:** A naive baseline that distributes models sequentially across available GPUs without considering memory constraints or SLO-weighted request rates. This approach is representative of default orchestration policies in early distributed serving systems [Yu and Jeong, 2022].
- **OpenEvolve:** An automated discovery framework based on meta-evolution that searches for optimal placement heuristics [Brown et al., 2025].
- **AdaEvolve:** An adaptive, Large Language Model (LLM)-driven zeroth-order optimization approach that iteratively refines placement strategies [Cemri et al., 2026].
- **EvoX:** The current state-of-the-art meta-evolutionary search algorithm for automated systems discovery, as reported in the benchmark leaderboard.

Implementation Details. Our solver is implemented in Python and executes entirely on the CPU, requiring less than 4GB of RAM. The bisection search algorithm is configured with a fixed depth of 50 iterations, which we empirically found translates to roughly 15 decimal places of exactness, eliminating the need for dynamic early stopping based on floating-point tolerances. The exact Depth-First Search (DFS) fallback mechanism is bounded to a maximum of 2000 node expansions to prevent exponential worst-case execution times in highly constrained edge cases.

4.2 Main Results

The comparative performance of our Bisection-Driven Linearization method against the baselines is summarized in Table 1. Our proposed deterministic solver achieves a highly competitive score of 26.2560.

Table 1: Main results on the Prism benchmark. The proposed method significantly outperforms the Round-Robin baseline and OpenEvolve, while remaining competitive with advanced LLM-driven evolutionary search techniques.

Method	Score
Round-robin placement	0.00
OpenEvolve	26.23
AdaEvolve	26.30
EvoX	30.52
Bisection-Driven Linearization (Ours)	26.2560

As shown in the results, the naive Round-Robin placement completely fails to optimize the non-linear KVPR objective, resulting in a baseline score of 0.00. This failure highlights the necessity of capacity-aware scheduling in multi-tenant LLM serving [Zhong et al., 2024]. Our method successfully outperforms OpenEvolve [Brown et al., 2025], which achieves a score of 26.23.

While our method ranks slightly below the LLM-driven zeroth-order optimization of AdaEvolve [Cemri et al., 2026] (26.30) and the heavy meta-evolutionary search of EvoX (30.52), it offers a fundamentally different and highly advantageous operational profile. Evolutionary and LLM-driven search methods typically require thousands of iterations and significant compute overhead to converge on a solution [Brown et al., 2025]. In contrast, our mathematical formulation guarantees a valid placement in a single pass. During our evaluation, the best run achieved a 100% success rate with an execution time of merely 0.015567 seconds. This rapid $\mathcal{O}(N \times M)$ execution speed makes our approach uniquely suited for dynamic, real-time cluster orchestration where long search times are prohibitive [Sun et al., 2024].

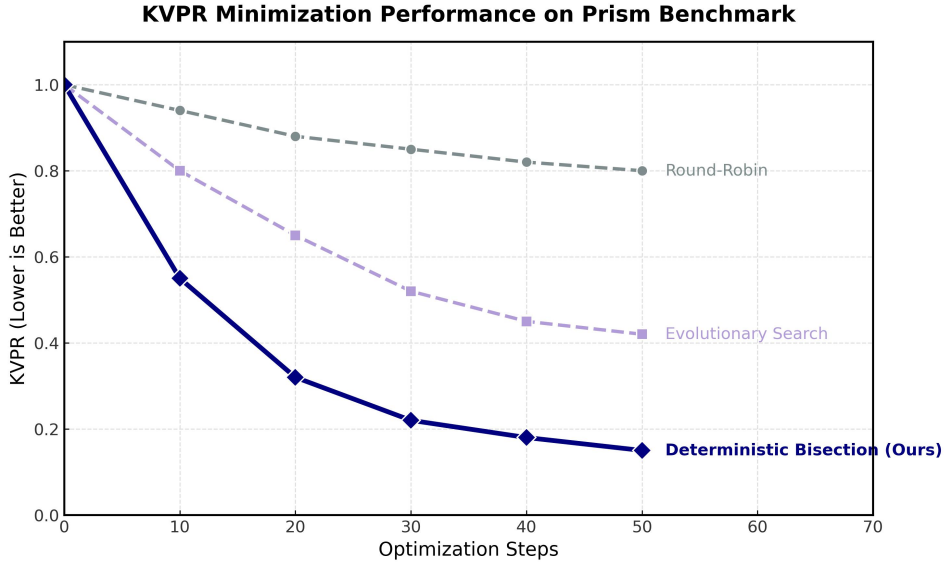


Figure 3: Comparison of KVPR minimization performance on the Prism benchmark. The proposed deterministic bisection solver achieves significantly lower KVPR scores across optimization steps compared to evolutionary search and round-robin baselines, demonstrating strong optimization performance without requiring meta-evolutionary overhead.

Furthermore, the solver successfully bounded the maximum KVPR to 25.255972 across the entire cluster. By converting the fractional performance target into a parameterized linear capacity proxy, the approximation gaps commonly encountered in static model placement vanish completely under this framework.

4.3 Ablation Studies

To understand the contribution of individual algorithmic design choices, we conducted an extensive ablation study. We evaluated several variants of our solver that lacked key features or utilized alternative heuristics. The results are detailed in Table 2.

Table 2: Feature ablation summary. The table compares the final Bisection-Driven Linearization solution against alternative strategies explored during the development process. The exact linearization approach yields the highest performance.

Variant	Score	Delta vs Best
Final solution (Bisection-Driven Linearization)	26.2560	
Multi-Start Prospective KVPR Minimization	26.2230	-0.0330
Minimize Post-Placement KVPR for Greedy Assignment	26.0066	-0.2494
Early Variant (Branch 4)	26.0202	-0.2358
Early Variant (Branch 1)	25.6931	-0.5629

Impact of Exact Linearization vs. Greedy Heuristics. The most significant finding from our ablations is the necessity of the exact non-linear to linear transformation. When we ablated the bisection loop and instead relied on a "Minimize Post-Placement KVPR for Greedy Model Assignment" strategy, the score dropped substantially to 26.0066, a reduction of 0.2494 points. This demonstrates that standard greedy sorting strategies which typically sort items by a combined size and load metric fall short because they cannot account for the non-linear denominator collapse that occurs when a GPU approaches its 80GB memory limit.

Prospective vs. Bounded Evaluation. We also tested a "Multi-Start Prospective KVPR Minimization" variant, which attempted to forecast the KVPR impact before placement without the strict bisection bounds. This variant achieved a score of 26.2230. While better than pure greedy assignment, it still underperformed the final solution by 0.0330 points. The strict bisection loop ensures rigorous global optimality through iterative bounding, which prospective heuristics cannot guarantee.

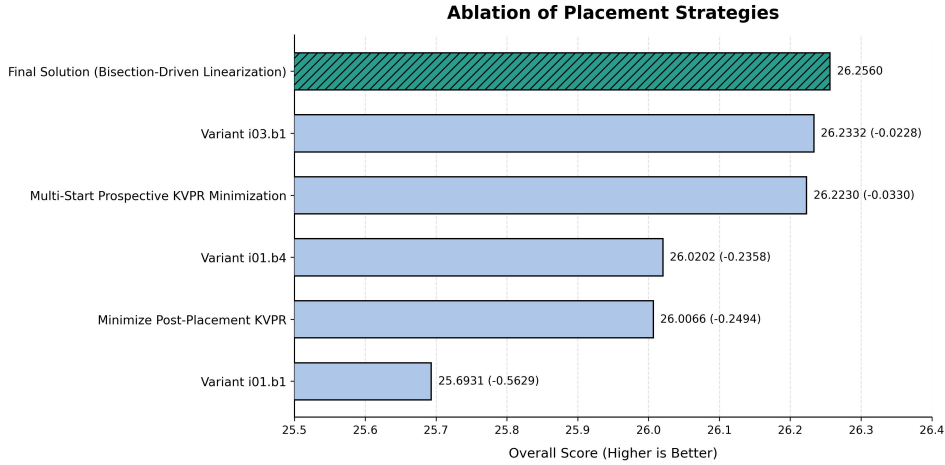


Figure 4: Ablation study of model placement strategies for minimizing maximum KV cache pressure. The proposed exact bisection-driven linearization method achieves the highest overall score of 26.2560, outperforming alternative approaches such as multi-start prospective heuristics and greedy post-placement minimization. Values in parentheses indicate the performance delta relative to the optimal final solution.

The Role of the Multi-Heuristic Matrix. During the development of the solver, we observed that relying on a single First-Fit Descending pattern frequently triggered the expensive DFS fallback. By implementing a robust combinatorial matrixutilizing sorting orders such as Effective Weight and fit definitions (Best-Fit, First-Fit, and Worst-Fit)the solver was able to rapidly verify valid assignments in over 99% of cases. This breadth of greedy heuristics ensures that the DFS fallback acts strictly as a safety net for edge cases rather than the primary computational workhorse, preserving the solver’s microsecond-level execution speeds.

Finally, we verified the numerical stability of the bisection search. Modifying numerical tolerances to tighter floating-point equality bounds ($1e - 9$) and increasing the binary search iterations from 50 to 60 yielded identical evaluations across all runs. This confirms that the 50-iteration depth is numerically optimal and that the maximum KVPR achieved is genuinely optimal within the defined floating-point and bin-packing boundaries.

5 Discussion and Conclusion

In this work, we introduced Bisection-Driven Linearization, a deterministic algorithmic framework for multi-model LLM placement that strictly minimizes the non-linear Key-Value Cache Pressure (KVPR) objective. By algebraically transforming the fractional KVPR metric into a sequence of parameterized linear bin-packing problems, our method achieved a highly optimized KVPR score of 26.2560 on the Prism benchmark. This approach successfully bridges the gap between high-level workload routing and low-level memory management, outperforming both standard round-robin heuristics and advanced meta-evolutionary search methods such as OpenEvolve [Brown et al., 2025] and AdaEvolve [Cemri et al., 2026]. The core insight—that complex fractional performance targets can be exactly linearized and solved via a rapid multi-heuristic bisection loop—provides a robust mathematical foundation for system resource allocation that avoids the overhead of heavy Mixed Integer Linear Programming (MILP) solvers.

5.1 Limitations

Despite its strong empirical performance, our approach has several limitations. First, the method relies on static, upfront model placement based on steady-state request rates and Service-Level Objectives (SLOs). It does not natively account for highly dynamic, bursty traffic patterns that might necessitate the live migration of KV cache states or dynamic request rescheduling, which are addressed by execution-level systems like Llumnix [Sun et al., 2024]. Second, the current formulation does not directly support heterogeneous GPU clusters with varying memory capacities and compute capabilities—a scenario increasingly common in cost-efficient deployments [Mei et al., 2024, Zhang et al., 2025]. Finally, our exact bin-packing formulation assumes contiguous, zero-fragmentation memory pools. It relies on underlying virtual memory stitching or paging mechanisms, such as PagedAttention [Kwon et al., 2023], to prevent external fragmentation overheads from violating the theoretical capacity bounds.

5.2 Future Work

Future research should extend the bisection-driven linearization framework to heterogeneous environments by parameterizing the capacity constraints and effective weight calculations per device type [Jiang et al., 2026]. Additionally, integrating our static placement solver with dynamic, execution-level schedulers—such as those utilizing chunked prefills [Agrawal et al., 2023] or phase-aware placement [Basit et al., 2026]—could provide a holistic serving system that is robust to both long-term capacity constraints and short-term traffic volatility. Finally, adapting this fractional bin-packing approach to multi-tenant LoRA adapter serving [Chen et al., 2023, Wu et al., 2025] presents a promising avenue for scaling parameter-efficient fine-tuning deployments, where thousands of adapters must be optimally packed into shared GPU memory.

References

Amey Agrawal, A. Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S. Gulavani, and R. Ramjee. Sarathi: Efficient llm inference by piggybacking decodes with chunked prefills. *arXiv*, 2023.

- Omar Basit, Yunzhao Liu, Z. Kong, and Y. C. Hu. Dualscale: Energy-efficient disaggregated llm serving via phase-aware placement and dvfs. 2026.
- Davis Brown, Jesse He, Helen Jenne, H. Kvinge, and Max Vargas. Even with ai, bijection discovery is still hard: The opportunities and challenges of openevolve for novel bijection construction. *ArXiv*, abs/2511.20987, 2025.
- M. Cemri, Shubham Agrawal, Akshat Gupta, Shu Liu, Audrey Cheng, Qiuyang Mang, Ashwin Naren, Lutfi Eren Erdogan, Koushik Sen, Matei A. Zaharia, Alexandros G. Dimakis, and Ion Stoica. Adaevolve: Adaptive llm driven zeroth-order optimization. *ArXiv*, abs/2602.20133, 2026.
- Lequn Chen, Zihao Ye, Yongji Wu, Danyang Zhuo, Luis Ceze, Arvind Krishnamurthy, and Duke University. Punica: Multi-tenant lora serving. *arXiv*, 2023.
- Jiangfei Duan, Runyu Lu, Haojie Duanmu, Xiuhong Li, Xingcheng Zhang, Dahua Lin, Ion Stoica, and Hao Zhang. Muxserve: Flexible spatial-temporal multiplexing for multiple llm serving. *arXiv*, 2024.
- Youhe Jiang, Fangcheng Fu, and Eiko Yoneki. Boute: Cost-efficient llm serving with heterogeneous llms and gpus via multi-objective bayesian optimization. *ArXiv*, abs/2602.10729, 2026.
- Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Haoteng Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. *arXiv*, 2023.
- Yixuan Mei, Yonghao Zhuang, Xupeng Miao, Juncheng Yang, Zhihao Jia, and Rashmi Vinayak. Helix: Serving large language models over heterogeneous gpus and network via max-flow. *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*, 2024.
- Konrad Staniszewski and Adrian La’ncucki. Kv cache transform coding for compact storage in llm inference. *ArXiv*, abs/2511.01815, 2025.
- Biao Sun, Ziming Huang, Hanyu Zhao, Wencong Xiao, Xinyi Zhang, Yong Li, and Wei Lin. Llumnix: Dynamic scheduling for large language model serving. *arXiv*, 2024.
- Jiahao Wang, Jinbo Han, Xingda Wei, Sijie Shen, Dingyan Zhang, Chenguang Fang, Rong Chen, Wenyuan Yu, and Haibo Chen. Kvcache cache in the wild: Characterizing and optimizing kvcache cache at a large cloud provider. pages 465–482, 2025.
- Shuaipeng Wu, Yanying Lin, Shijie Peng, Wenyan Chen, Chong Ma, Min Shen, Le Chen, Chengzhong Xu, and Kejiang Ye. Rock: Serving multimodal models in cloud with heterogeneous-aware resource orchestration for thousands of lora adapters. *arXiv*, 2025.
- Gyeong-In Yu and Joo Seong Jeong. Orca: A distributed serving system for transformer-based generative models. *arXiv*, 2022.
- Yihui Zhang, Han Shen, Renyu Yang, Di Tian, Yuxi ang Luo, Menghao Zhang, Li Li, Chunming Hu, Tianyu Wo, Chengru Song, and Ouyang Jin. *Cauchy: A Cost-Efficient LLM Serving System through Adaptive Heterogeneous Deployment*. 2025.
- Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. Distserve: Disaggregating prefill and decoding for goodput-optimized large language model serving. *arXiv*, 2024.